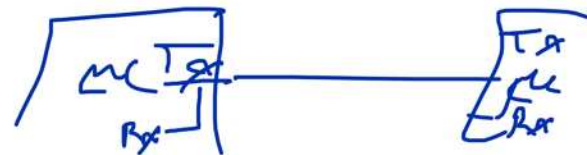
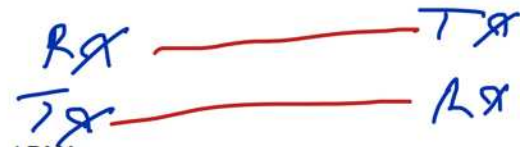
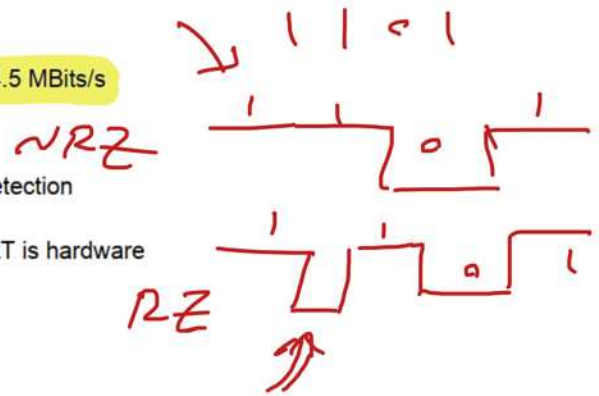


27.2

USART main features

- Full duplex, asynchronous communications
- NRZ standard format (Mark/Space)
- Fractional baud rate generator systems
 - A common programmable transmit and receive baud rates up to 4.5 MBits/s
- Programmable data word length (8 or 9 bits)
- Configurable stop bits - support for 1 or 2 stop bits
- LIN Master Synchronous Break send capability and LIN slave break detection capability
 - 13-bit break generation and 10/11 bit break detection when USART is hardware configured for LIN
- Transmitter clock output for synchronous transmission
- IrDA SIR Encoder Decoder
 - Support for 3/16 bit duration for normal mode
- Smartcard Emulation Capability
 - The Smartcard interface supports the asynchronous protocol Smartcards as defined in ISO 7816-3 standards
 - 0.5, 1.5 Stop Bits for Smartcard operation
- Single wire half duplex communication
- Configurable multibuffer communication using DMA (direct memory access)
 - Buffering of received/transmitted bytes in reserved SRAM using centralized DMA
- Separate enable bits for Transmitter and Receiver
- Transfer detection flags:
 - Receive buffer full
 - Transmit buffer empty
 - End of Transmission flags
- Parity control:
 - Transmits parity bit
 - Checks parity of received data byte
- Four error detection flags:
 - Overrun error
 - Noise error
 - Frame error
 - Parity error
- Ten interrupt sources with flags:
 - CTS changes
 - LIN break detection
 - Transmit data register empty
 - Transmission complete
 - Receive data register full
 - Idle line received
 - Overrun error
 - Framing error
 - Noise error
 - Parity error
- Multiprocessor communication - enter into mute mode if address match does not occur
- Wake up from mute mode (by idle line detection or address mark detection)
- Two receiver wakeup modes: Address bit (MSB, 9th bit), Idle line



RS485

Handshake

RS232

Figure 279. USART block diagram

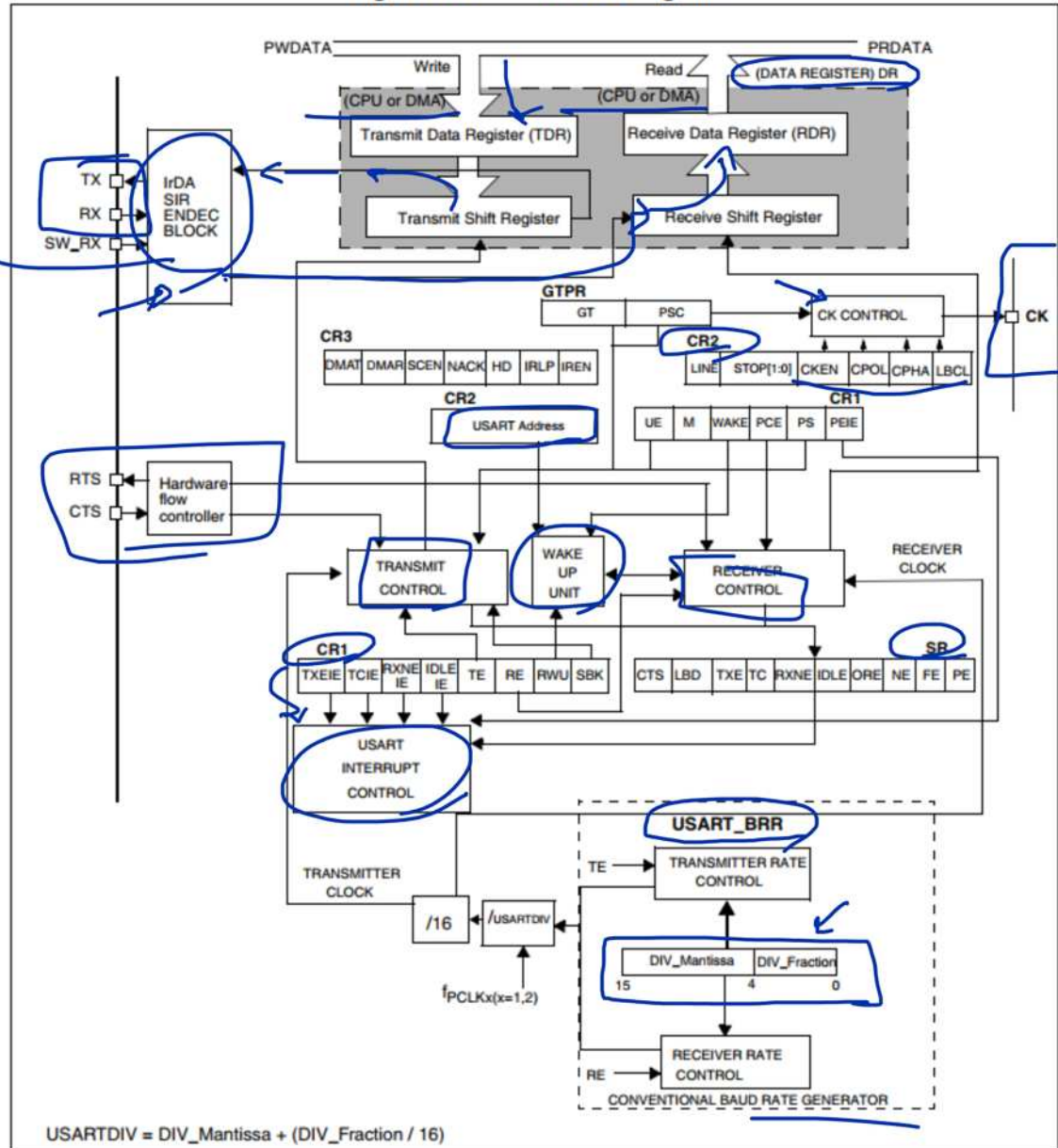
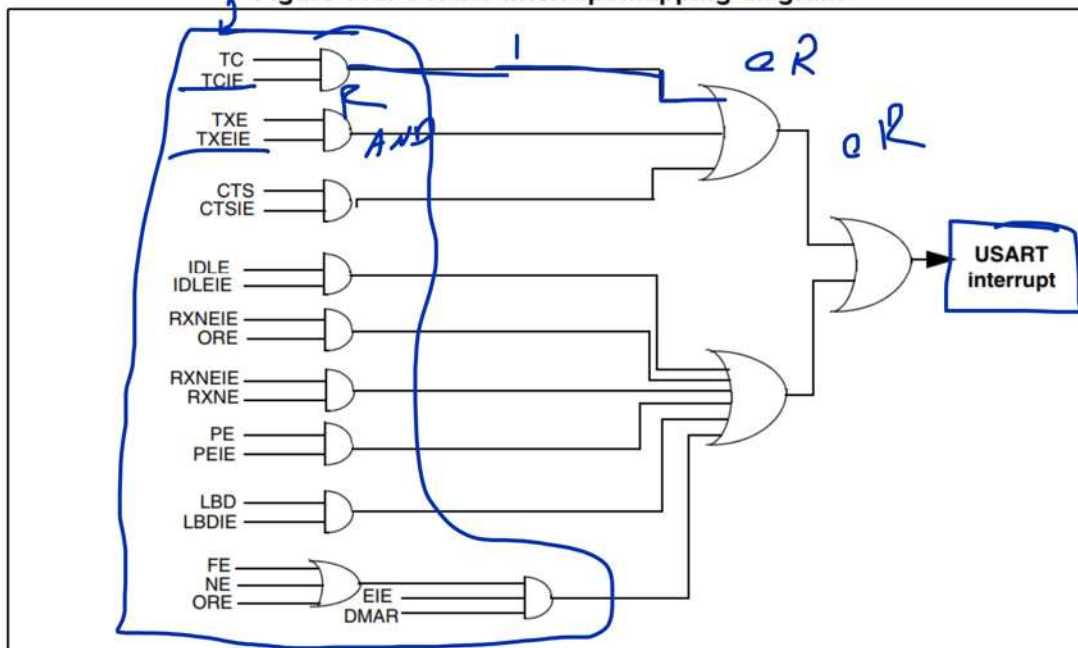


Figure 302. USART interrupt mapping diagram



27.3.4 Fractional baud rate generation

The baud rate for the receiver and transmitter (Rx and Tx) are both set to the same value as programmed in the Mantissa and Fraction values of USARTDIV.

$$\text{Tx/Rx baud} = \frac{f_{\text{CK}}}{16 \times \text{USARTDIV}}$$

legend: f_{CK} - Input clock to the peripheral (PCLK1 for USART2, 3, 4, 5 or PCLK2 for USART1)

PCLK2 $\rightarrow$ 1 64MHz
 $\rightarrow$ 2 } PCLK1
 $\rightarrow$ 3 }

USARTDIV is an unsigned fixed point number that is coded on the USART_BRR register.

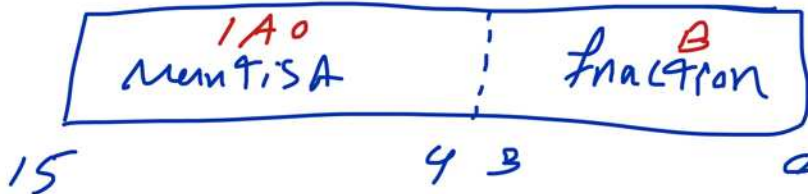
Note: The baud counters are updated with the new value of the Baud registers after a write to USART_BRR. Hence the Baud rate register value should not be changed during communication.

$$\text{Div} = \frac{f}{16 \times B} \quad \left| \quad B = 9600 \right. \quad = \frac{64 \times 10^6}{16 \times 9600}$$

$$\text{Div} \approx 416.667$$

Mantissa = 416
 Fraction = 0.667

$$\text{Fraction} \approx 0.667 \times 16 = 10.667 \Rightarrow \text{fraction} = 11$$



$\Rightarrow$ BRR = 0x1A0B

$$\text{Div} = \frac{64 \times 10^6}{16 \times 115200} = 34.722$$

Mantissa = 34
 Fraction = 0.722

$$0.722 \times 16 = 11.55 \Rightarrow \text{fraction} = 12 \rightarrow 0xC$$

0x22 | 0xC $\Rightarrow$ BRR = 0x22C

$$\text{BRR} = \text{Mantissa} \times 16 + \text{Fraction} \times 16 = 16 (\text{Mantissa} + \text{Fraction})$$

$$\text{BRR} = 16 \left(\frac{f_{\text{CLK}}}{16 \times B} \right) = \frac{f_{\text{CLK}}}{B} = \frac{64 \times 10^6}{115200} \Rightarrow 556$$

HAL_UART_Transmit

Function name

HAL_StatusTypeDef HAL_UART_Transmit (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Sends an amount of data in blocking mode.

Parameters

- **huart**: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pData.

HAL_USART_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_USART_Transmit_DMA (USART_HandleTypeDef * husart, uint8_t * pTxData, uint16_t Size)

Function description

Simplex Send an amount of data in DMA mode.

Parameters

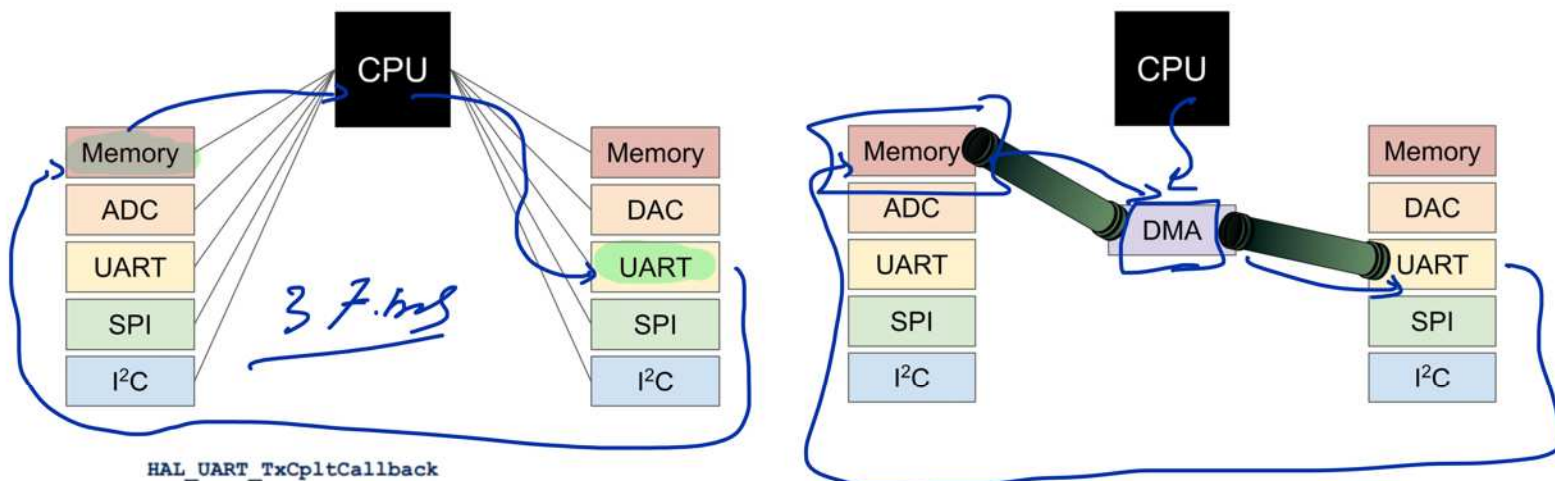
- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent.

Return values

- **HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pTxData.



HAL_UART_TxCpltCallback

Function name

void HAL_UART_TxCpltCallback (UART_HandleTypeDef * huart)

Function description

Tx Transfer completed callbacks.

Parameters

- **huart**: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None**:

HAL_UART_Receive_IT

Function name

HAL_StatusTypeDef HAL_UART_Receive_IT (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Receives an amount of data in non blocking mode.

Parameters

- **huart**: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be received.

Return values

- **HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pData.

Function name

void HAL_UART_RxCpltCallback (UART_HandleTypeDef * huart)

Function description

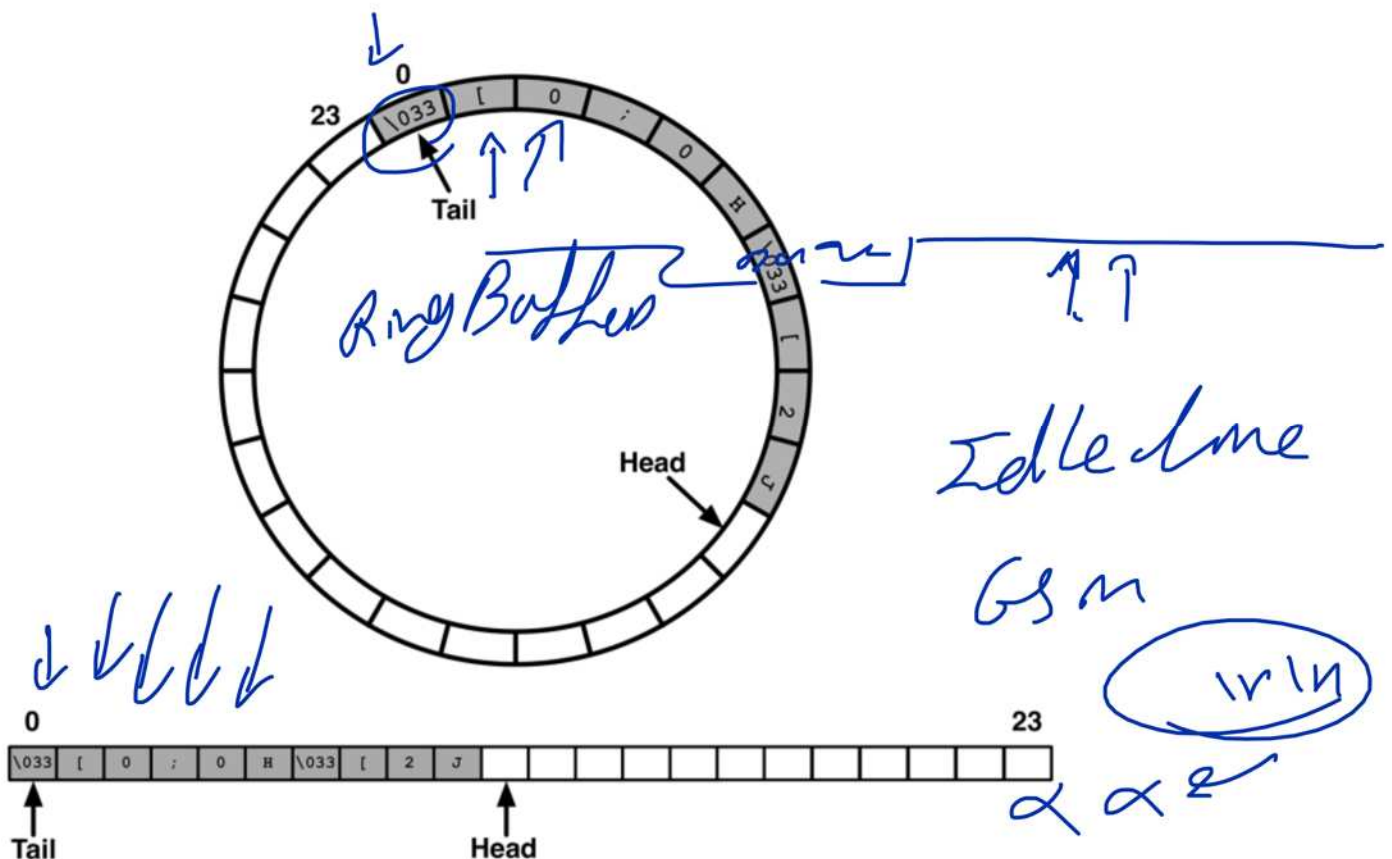
Rx Transfer completed callbacks.

Parameters

- **huart**: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None**:




```

/**
 * @brief Receive an amount of data in interrupt mode till either the expected number of data is received or an IDLE event occurs.
 * @note Reception is initiated by this function call. Further progress of reception is achieved thanks
 * to UART interrupts raised by RXNE and IDLE events. Callback is called at end of reception indicating
 * number of received data elements.
 * @note When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M = 01),
 * the received data is handled as a set of uint16_t. In this case, Size must indicate the number
 * of uint16_t available through pData.
 * @param huart UART handle.
 * @param pData Pointer to data buffer (uint8_t or uint16_t data elements).
 * @param Size Amount of data elements (uint8_t or uint16_t) to be received.
 * @retval HAL status
 */
HAL_StatusTypeDef HAL_UARTEx_ReceiveToIdle_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)

/**
 * @brief Receive an amount of data in DMA mode till either the expected number of data is received or an IDLE event occurs.
 * @note Reception is initiated by this function call. Further progress of reception is achieved thanks
 * to DMA services, transferring automatically received data elements in user reception buffer and
 * calling registered callbacks at half/end of reception. UART IDLE events are also used to consider
 * reception phase as ended. In all cases, callback execution will indicate number of received data elements.
 * @note When the UART parity is enabled (PCE = 1), the received data contain
 * the parity bit (MSB position).
 * @note When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M = 01),
 * the received data is handled as a set of uint16_t. In this case, Size must indicate the number
 * of uint16_t available through pData.
 * @param huart UART handle.
 * @param pData Pointer to data buffer (uint8_t or uint16_t data elements).
 * @param Size Amount of data elements (uint8_t or uint16_t) to be received.
 * @retval HAL status
 */
HAL_StatusTypeDef HAL_UARTEx_ReceiveToIdle_DMA(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)

/**
 * @brief Reception Event Callback (Rx event notification called after use of advanced reception service).
 * @param huart UART handle
 * @param Size Number of data available in application reception buffer (indicates a position in
 * reception buffer until which, data are available)
 * @retval None
 */
__weak void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)

```

```
void HAL_UART_ErrorCallback (UART_HandleTypeDef * huart)
```

Function description

UART error callbacks.

Parameters

- **huart**: Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None**:

HAL_UART_AbortCpltCallback

```
HAL_StatusTypeDef HAL_UART_Abort (UART_HandleTypeDef * huart)
```

Function description

Abort ongoing transfers (blocking mode).

Parameters

- **huart**: UART handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx and Rx) Disable the DMA transfer in the peripheral register (if enabled) Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode) Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.